Biblioteka *Pandas*

Zestaw ćwiczeń do samodzielnej pracy

Spis treści

1. Wprowadzenie ... 3
1.1. Instalacja i import pakietu ... 3
2. Podstawowe struktury danych ... 4
2.1. Serie ... 4
2.2. Ramki danych ... 8
2.2.1. `DataFrame` jako uogólniona tablica *NumPy* ... 8
2.2.2. `DataFrame` jako specjalizowany słownik ... 10
2.2.3. Konstruowanie obiektów `DataFrame` ... 10
2.3. Indeksy ... 15
3. Konwencje i sposoby indeksowania ... 16
3.1. Indeksatory ... 18
3.2. Filtrowanie danych ... 20
4. Operowanie danymi ... 21
4.1. Operacje plikowe ... 21
4.1.1. Pliki CSV ... 21
4.1.2. Inne formaty ... 26
4.2. Wybrane funkcje i metody ... 26
5. Zadania sprawdzające ... 36
5.1. Podstawy pracy z ramkami danych ... 36
5.2. Operacje plikowe, konwersja danych ... 37

1. Wprowadzenie

Biblioteka *pandas*[1] zawiera zaawansowane struktury danych i narzędzia do manipulacji danymi zaprojektowane tak, aby przechowywanie, obróbka i analiza danych w *Pythonie* były szybkie i wygodne. Pakiet *pandas* jest często używany w połączeniu z narzędziami do obliczeń numerycznych, takimi jak *NumPy*[2] i *SciPy*[3], bibliotekami analitycznymi, takimi jak *statsmodels*[4] i *scikit-learn*[5], oraz bibliotekami do wizualizacji danych, takimi jak *matplotlib*[6], czy *seaborn*[7]. Moduł *pandas* przejmuje znaczną część idiomatycznego stylu obliczeń opartych na tablicach *NumPy* (zwłaszcza funkcji) i preferuje przetwarzanie danych bez pętli `for`. W połączeniu z *NumPy*, pakiet ten stanowi podstawę realizacji różnych scenariuszy uczenia maszynowego i *Data Science* – stąd jego znajomość i umiejętność efektywnego wykorzystania jest niezbędna do rozwijania swoich umiejętności w tych dziedzinach.

Pandas został wprawdzie zbudowany na bazie biblioteki *NumPy* i przyjmuje z niej wiele idiomów kodowania – jest jednak nowszy, operujący na wyższym poziomie abstrakcji – a co za tym idzie, wygodniejszy w użyciu, zapewniający wydajną implementację takich struktur danych, jak serie (`Series`) i ramki danych (`DataFrame`). Te ostatnie to zasadniczo tablice wielowymiarowe z dołączonymi etykietami wierszy i kolumn, często z heterogenicznymi ich typami i/lub brakującymi danymi – podczas gdy *NumPy* najlepiej nadaje się do pracy z jednorodnie typowanymi danymi w formie tablic numerycznych[8]. Oprócz wygodnego interfejsu przechowywania danych oznaczonych etykietami, *pandas* implementuje szereg zaawansowanych operacji na danych, znanych użytkownikom zarówno struktur baz danych, jak i programów arkuszy kalkulacyjnych.

1.1. Instalacja i import pakietu

Ćwiczenie 1: Instalacja pakietu *pandas*
Zainstaluj pakiet *pandas* w swoim środowisku pracy.

Sposób zarówno instalacji, jak i importu modułu *pandas* nie odbiega od tych dla większości bibliotek *Pythona* – jeżeli korzystamy z instalatora pakietów *PIP*, w terminalu wpisujemy polecenie:

```
pip install pandas
```

– jeżeli zaś z programu *Anaconda*:

```
conda install pandas
```

Ćwiczenie 2: Importowanie pakietu *pandas*
Zaimportuj pakiet *pandas* do swojego programu.

Po zainstalowaniu biblioteki, aby móc z niej korzystać w skrypcie/programie *Pythonowym*, importujemy ją instrukcją

```
import pandas as pd
```

Tradycyjnie nadając modułowi *pandas* alias `pd`

Ćwiczenie 3: Weryfikacja wersji pakietu *pandas*
Sprawdź wesję pakietu *pandas*, zainstalowaną w swoim środowisku kodu.

[1] https://pandas.pydata.org/
[2] https://numpy.org/
[3] https://scipy.org/
[4] https://www.statsmodels.org/stable/index.html
[5] https://scikit-learn.org/stable/index.html
[6] https://matplotlib.org/
[7] https://seaborn.pydata.org/
[8] *Nota bene*, będziemy mieli z nimi często do czynienia, używając pakietu `scikit-learn`

Pakiet *pandas* jest bodajże jednym z najintensywniej rozwijanych spośród jakże szerokiej gamy bibliotek *Pythona*. Najnowsza jego gałąź rozwojowa (2.2.x) została zaprezentowana w styczniu 2024 roku. Jest o tyle istotne, że do pakietu w kolejnych jego wydaniach nie tylko dodawane są nowe funkcjonalności, ale przede wszystkim zmieniane pewne konwencje, dotyczące np. zachowania pewnych funkcji, zwłaszcza wartości domyślnych ich argumentów. Stąd pojawiające się niekiedy ostrzeżenia o wycofywaniu w najbliższej przyszłości niektórych opcji lub zmianie ich znaczenia.

Warto więc pamiętać o sięganiu zawsze do dokumentacji tej wersji biblioteki, z którą mamy do czynienia w naszym środowisku pracy:

```
print(pd.__version__)
```

```
2.2.3
```

Ćwiczenie 4: Aktualizacja wersji pakietu *pandas*
Zaktualizuj pakiet *pandas*, zainstalowany w swoim środowisku kodu do najnowszej jego wersji.

O ile środowisko na to pozwala (nie wystąpią konflikty z innymi, zainstalowanymi juz w nim pakietami), a nowe lub zmodyfikowane funkcjonalności w kolejnej wersji *pandas*-a ułatwią nam wykonywanie zadań i przyspieszą kodowanie – warto wykonać aktualizację tej biblioteki do najnowszej wersji, wykonując w terminalu poniższe polecenie (odpowiednio dla *PIP*-a / *Anacondy*):

```
pip install pandas --upgrade
```

lub

```
conda install pandas --upgrade
```

2. Podstawowe struktury danych

Na bardzo podstawowym poziomie obiekty *pandasa* można traktować jako ulepszone wersje tzw. tablic strukturalnych pakietu *NumPy*, w których wiersze i kolumny są identyfikowane za pomocą etykiet, a nie prostych indeksów całkowitych. Jak zobaczymy w kolejnych podrozdziałach, *pandas* – oprócz podstawowych struktur danych – zapewnia szereg przydatnych narzędzi, metod i funkcjonalności do ich przetwarzania. Jednak, aby o nich mówić, trzeba najpierw zrozumieć, czym są owe struktury. Dlatego – zanim przejdziemy dalej – przedstawmy trzy podstawowe struktury danych *pandasa*: serię (`Series`), ramkę danych (`DataFrame`) i indeks (`Index`)[9].

2.1. Serie

Ćwiczenie 5: Tworzenie serii danych
Utwórz serię danych o nazwie `data`, bazując na liście `[0.25, 0.5, 0.75, 1.0]`. Wypisz tę serię na standardowe wyjście.

Seria (`Series`) to jednowymiarowa tablica indeksowanych danych. Można ją utworzyć z listy lub tablicy w następujący sposób:

```
data = pd.Series([0.25, 0.5, 0.75, 1.0])
```

A następnie wyświetlić:

```
print(data)
```

[9] Przykłady (dostosowując je) zaczerpnięto m. in. z podręczników: *Python Data Science Handbook* autorstwa Jake VanderPlas-a (https://jakevdp.github.io/PythonDataScienceHandbook/) oraz *Python for Data Analysis*, której autorem jest Wes McKinney, współtwórca pakietu *pandas* (https://wesmckinney.com/book/).

```
0    0.25
1    0.50
2    0.75
3    1.00
dtype: float64
```

Ćwiczenie 6: Struktura serii danych
Wypisz na standardowe wyjście oddzielnie zbiór wartości oraz indeksy powyższej serii.

Jak widzieliśmy w poprzednim ćwiczeniu, seria zawiera zarówno sekwencję wartości określonego typu (w naszym wypadku jest to 64-bitowy typ zmiennoprzecinkowy `float64`), jak i sekwencję indeksów, do których możemy uzyskać dostęp, odpowiednio za pomocą atrybutów `values`[10] oraz `index`. Ten pierwszy reprezentuje tablicę *NumPy*, za pomocą której seria przechowuje swoje wartości:

```
print(data.values)
```

```
array([ 0.25, 0.5 , 0.75, 1. ])
```

Atrybut `index` jest swoistą jednowymiarową tablicą klasy `Index`, o której w bliższych szczegółach powiemy nieco później:

```
print(data.index)
```

```
RangeIndex(start=0, stop=4, step=1)
```

Ponieważ nie określiliśmy w sposób jawny indeksu dla danych, tworzony jest (niejawnie) **domyślny indeks** klasy `RangeIndex`, składający się z liczb całkowitych od 0 do *N*–1 (gdzie *N* jest długością serii).

Ćwiczenie 7: Indeksowanie serii
Wypisz na standardowe wyjście element serii `data` o indeksie 1.

Podobnie jak w przypadku listy, czy tablicy *NumPy*, dostęp do danych można uzyskać poprzez powiązany z nimi indeks, za pomocą znanej w języku *Python* notacji z użyciem nawiasów kwadratowych:

```
print(data[1])
```

```
0.5
```

Zwróć uwagę, że numerowanie elementów serii indeksami niejawnymi rozpoczyna się od zera – podobnie, jak i dla innych kolekcji w języku *Python*.

Ćwiczenie 8: Wybieranie fragmentów serii
Wypisz na standardowe wyjście fragment serii `data`, zawarty pomiędzy indeksami 1 a 3.

Możemy w tym wypadku użyć znanego nam już z operacji na listach tzw. *slicingu*[11], aby wybrać fragment serii:

```
print(data[1:3])
```

```
1    0.50
2    0.75
dtype: float64
```

[10] Dokumentacja zaleca stosowanie metody `to_numpy()`, oferującej większą elastyczność w konwersji danych lub atrybutu `array`, dającego dostęp do tzw. tablic typów rozszerzonych, bardziej uniwersalnych niż tablice *NumPy*. Użycie atrybutu `values` podyktowane jest dopasowaniem typów danych do potrzeb pakietu *scikit-learn*.

[11] W języku angielskim oznacza to „krojenie na plasterki”, co obrazowo oddaje wykonywaną na kolekcji czynność.

Warto przypomnieć, że – podobnie jak to ma miejsce dla list – indeksy[12] serii zaczynają się od zera, a zatem górny zakres *slicingu* (fragmentacji) brany jest ekskluzywnie (wyłącznie).

Ćwiczenie 9: Tworzenie serii danych o jawnym indeksie
Utwórz serię danych `data`, bazując na liście `[0.25, 0.5, 0.75, 1.0]`, nadając jej elementom indeksy z listy `['c', 'b', 'f', 'd']`. Wypisz tę serię na standardowe wyjście.

Z tego, co widzieliśmy do tej pory, mogłoby się wydawać, że obiekt `Series` da się w zasadzie zastąpić jednowymiarową tablicą *NumPy*. Podstawową różnicą jest jednak obecność indeksu: podczas gdy tablica *NumPy* ma wyłącznie domyślnie zdefiniowany, zaczynający się od zera indeks całkowity, używany w celu uzyskania dostępu do wartości, seria *pandasa* może mieć też **jawnie** zdefiniowany indeks, powiązany z wartościami, co zapewnia obiektowi `Series` dodatkowe możliwości. Nie musi on być liczbą całkowitą, ale może składać się z wartości dowolnego typu. Na przykład, jeśli chcemy, możemy użyć łańcuchów znaków jako indeksu (nie muszą one stanowić zwartej sekwencji):

```
data = pd.Series([0.25, 0.5, 0.75, 1.0], index=['c', 'b', 'f', 'd'])
print(data)
```

```
c    0.25
b    0.50
f    0.75
d    1.00
dtype: float64
```

Ćwiczenie 10: Indeksowanie jawne serii
Wypisz na standardowe wyjście element powyższej serii o indeksie `'b'`.

Mając serię, indeksowaną jawnie, dostęp do jej elementów możliwy jest następująco:

```
print(data['b'])
```

```
0.5
```

Ćwiczenie 11: Wybieranie fragmentów serii za pomocą indeksów jawnych
Wypisz na standardowe wyjście fragment serii `data`, zawarty pomiędzy indeksami `'b'` a `'d'`.

Zwróć uwagę, że w przypadku wyspecyfikowania indeksów jawnych, przy wycinaniu fragmentu serii (*slicingu*) górny zakres brany jest inkluzywnie (włącznie):

```
print(data['b':'d'])
```

```
b    0.50
f    0.75
d    1.00
dtype: float64
```

Ćwiczenie 12: Modyfikowanie elementów serii
Nadaj elementowi serii `data` o indeksie `'c'` wartość 3.14. Wyswietl zmodyfikowaną serię.

W oczywisty sposób, indeksów (czy to jawnych, czy niejawnych) można – analogicznie, jak dla list – użyć do modyfikowania danych:

```
data['c'] = 3.14
print(data)
```

[12] Są to tzw. indeksy niejawne, automatycznie tworzone domyślnie przy konstruowaniu serii, czy ramek danych. W przypadku indeksów specyfikowanych jawnie, zasady slicingu są nieco inne, o czym za chwilę.

```
c    3.14
b    0.50
f    0.75
d    1.00
dtype: float64
```

Ćwiczenie 13: Specyfikowanie atrybutów serii
Utwórz serię danych, bazując na liście `[2, 3, 5, 7, 11]`, nadając jej nazwę `my_data` i określając typ danych jej elementów jako `float32`. Wypisz tak otrzymaną serię na standardowe wyjście.

Używając konstruktora klasy `Series`, można wyspecyfikować szereg jego argumentów (takich, jak użyty przed chwilą `index`), modyfikujących sposób tworzenia serii. Pozwala to w szczególności określić np. nazwę serii[13] (argument `name`), czy typ danych (argument `dtype`), w niej zawartych – o ile ma on być inny, niż wynikający z typu wartości, na bazie których tworzymy obiekt `Series`:

```
data = pd.Series([2, 3, 5, 7, 11], name='my_data', dtype='float32')
print(data)
```

```
0    2.0
1    3.0
2    5.0
3    7.0
4   11.0
Name: my_data, dtype: float32
```

Jak widać, mimo iż wartości na liście, służącej do zdefiniowania serii są całkowite, użycie argumentu `dtype` spowodowało, że zawiera ona 32-bitowe dane zmiennoprzecinkowe.

Myśląc w ten sposób, można traktować serie *pandasa* jak specjalizowane słowniki *Pythona*. Słownik to struktura, która odwzorowuje dowolne klucze na zestaw dowolnych wartości, a seria to struktura, która odwzorowuje indeksy danego typu na zestaw wartości (również określonego typu).

Typowanie to jest bardzo istotne: tak jak skompilowany kod, specyficzny dla typu danych tablicy *NumPy* sprawia, że jest ona zazwyczaj bardziej wydajna niż lista *Pythona*, tak informacje o typie danych serii *pandasa* sprawiają, że jest ona w przypadku większości operacji znacznie wydajniejsza, niż słowniki *Pythona*.

Analogię serii jako słownika można uczynić jeszcze bardziej przejrzystą, konstruując obiekt `Series` bezpośrednio ze słownika *Pythona*.

Ćwiczenie 14: Seria danych jako słownik
Korzystając z poniższej tabeli, zawierającej informację o liczbie ludności w kilku wybranych stanach USA, utwórz (a następnie wyświetl) serię `population` dla tych danych, indeksowaną nazwami stanów. Użyj metody słownikowej tworzenia serii.

State	Population
California	38332521
Texas	26448193
New York	19651127
Florida	19552860
Illinois	12882135

[13] Nie chodzi tu bynajmniej o nazwę referencji, którą pokazujemy na obiekt klasy `Series` a o związany z nim atrybut klasy.

Utworzymy najpierw słownik, posiadający jako klucze nazwy poszczególnych stanów, zaś jako wartości – liczbę ludności w tych stanach. Następnie użyjemy go jako argumentu konstruktora ramki danych:

```
population_dict = {'California': 38332521,
                   'Texas': 26448193,
                   'New York': 19651127,
                   'Florida': 19552860,
                   'Illinois': 12882135}
population = pd.Series(population_dict)
print(population)
```

```
California    38332521
Texas         26448193
New York      19651127
Florida       19552860
Illinois      12882135
dtype: int64
```

2.2. Ramki danych

Następną podstawową strukturą *pandasa* jest ramka danych (`DataFrame`). Podobnie jak obiekt `Series`, omówiony w poprzedniej sekcji, ramkę danych można traktować jako uogólnienie tablicy *NumPy* lub jako specjalizację słownika *Pythona*. Przyjrzyjmy się teraz każdej z tych perspektyw.

2.2.1. `DataFrame` jako uogólniona tablica *NumPy*

Podobnie jak seria jest analogiem jednowymiarowej tablicy z elastycznymi indeksami, tak ramka danych jest analogiem dwuwymiarowej tablicy z elastycznymi indeksami wierszy i elastycznymi nazwami kolumn. Podobnie jak tablicę dwuwymiarową można postrzegać jako uporządkowaną sekwencję jednowymiarowych kolumn tej samej długości, tak ramkę danych można traktować jako sekwencję dopasowanych (tzn. mających ten sam indeks) serii.

Ćwiczenie 15: Seria danych jako słownik (c.d.)
Korzystając z poniższej tabeli, zawierającej informację o polu powierzchni stanów USA (tych samych, co w poprzednim ćwiczeniu), utwórz (a następnie wyświetl) serię `area` dla tych danych, indeksowaną nazwami stanów. Użyj metody słownikowej tworzenia serii.

State	Area
California	423967
Texas	695662
New York	141297
Florida	170312
Illinois	149995

Postępujemy analogicznie, jak w poprzednim ćwiczeniu:

```
area_dict = {'California': 423967,
             'Texas': 695662,
             'New York': 141297,
             'Florida': 170312,
             'Illinois': 149995}
area = pd.Series(area_dict)
print(area)
```

```
California    423967
Texas         695662
New York      141297
Florida       170312
Illinois      149995
dtype: int64
```

Ćwiczenie 16: Tworzenie ramki danych z serii
Biorac serię z poprzedniego ćwiczenia, razem z wcześniej utworzoną serią z danymi o liczbie ludności, użyj słownika do skonstruowania dwuwymiarowego obiektu `DataFrame` o nazwie `states`, zawierającego te informacje. Wyświetl otrzymaną ramkę danych oraz informacje o niej.

Tworząc ramkę danych, jako argument jej konstruktora podamy słownik, którego kluczami będą nazwy kolumn ramki danych, zaś wartościami – wcześniej utworzone serie:

```
states = pd.DataFrame({'population': population, 'area': area})
print(states)
```

```
            population    area
California    38332521  423967
Texas         26448193  695662
New York      19651127  141297
Florida       19552860  170312
Illinois      12882135  149995
```

```
states.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 5 entries, California to Illinois
Data columns (total 2 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   population  5 non-null      int64
 1   area        5 non-null      int64
dtypes: int64(2)
```

Ćwiczenie 17: Struktura ramki danych
Wypisz na standardowe wyjście listę indeksów wierszowych oraz (oddzielnie) listę indeksów kolumnowych (nazw kolumn) ramki danych z poprzedniego ćwiczenia. Następnie wyświetl tablicę *NumPy*, przechowującą wartości danych w tej ramce.

Podobnie jak obiekt `Series`, ramka danych posiada atrybut `index`, który zapewnia dostęp do etykiet wspólnego indeksu wierszy:

```
print(states.index)
```

```
Index(['California', 'Texas', 'New York', 'Florida', 'Illinois'], dtype='object')
```

Ponadto ramka danych ma atrybut `columns`, będący również obiektem typu `Index`, ale – jak sama nazwa wskazuje – przechowującym etykiety (nazwy) kolumn:

```
print(states.columns)
```

```
Index(['population', 'area'], dtype='object')
```

Ramkę danych (obiekt `DataFrame`) można traktować jako rozszerzenie dwuwymiarowej tablicy *NumPy*, w której zarówno wiersze, jak i kolumny posiadają uogólniony indeks. Użycie na rzecz tego obiektu metody `to_numpy()` lub atrybutu `values` zwróci tablicę *NumPy*, przechowującą wartości ramki danych:

```
print(states.to_numpy())
```

```
[[38332521   423967]
 [26448193   695662]
 [19651127   141297]
 [19552860   170312]
 [12882135   149995]]
```

2.2.2. `DataFrame` jako specjalizowany słownik

Alternatywnie możemy również myśleć o ramce danych jako o specjalizowanym słowniku. Tak, jak słownik odwzorowuje klucz na wartość, tak `DataFrame` odwzorowuje nazwę kolumny na serię danych kolumny (w formie `ramka['kolumna']`), zwracając obiekt `Series`, zawierający dane z tejże kolumny. Nazwa kolumny ramki danych może być również traktowana jako atrybut obiektu `DataFrame`, zatem możemy się do niej odwołać także w formie `ramka.kolumna`. Jednak ten sposób zapisu nie jest zalecany, gdyż ma kilka ograniczeń – nie można go użyć, gdy nazwa kolumny zawiera spację lub znaki spoza zestawu ASCII/UTF-8, jest tożsama z nazwą wbudowanego atrybutu bądź metody klasy `DataFrame`, a także wówczas, gdy tworzymy nową kolumnę w istniejącej ramce danych.

Ćwiczenie 18: Wybieranie kolumny z ramki danych
Wyświetl pola powierzchni poszczególnych stanów, zawartych w ramce danych `states`.

Nazwę kolumny `area` traktujemy jako indeks (kolumnowy) ramki danych:

```
print(states['area'])
```

```
California    423967
Texas         695662
New York      141297
Florida       170312
Illinois      149995
Name: area, dtype: int64
```

Zwróć uwagę na tkwiące w tej konwencji potencjalne źródło nieporozumień: w przypadku dwuwymiarowej tablicy *NumPy* wyrażenie `data[0]` zwróci pierwszy wiersz – natomiast dla ramki `DataFrame`, analogiczne wyrażenie `data['col0']` zwróci pierwszą kolumnę. Z tego powodu lepiej jest myśleć o ramkach danych jako o słownikach, a nie o tablicach, chociaż oba sposoby patrzenia na sytuację mogą być przydatne. Bardziej elastycznym sposobom indeksowania ramek danych przyjrzymy się w kolejnych rozdziałach.

2.2.3. Konstruowanie obiektów `DataFrame`

Ramkę danych *pandasa* można skonstruować na różne sposoby – ponieważ, jak już wiemy, jest ona zbiorem serii o wspólnym indeksie wierszy, to jednokolumnowy obiekt `DataFrame` może być zbudowany również z pojedynczego obiektu `Series`.

Ćwiczenie 19: Tworzenie jednokolumnowej ramki danych z serii
Skonstruuj ramkę danych `df_area`, składającą się z jednej kolumny, zawierającej dane z serii `area`. Następnie wypisz i porównaj informacje o typie danych utworzonej ramki danych oraz typie danych serii `area`.

W celu wyspecyfikowania (jednoelementowej) listy nazw (etykiet, indeksów) kolumn użyjemy argumentu `columns` konstruktora klasy `DataFrame`:

```
df_area = pd.DataFrame(area, columns=['area'])
print(df_area)
```

```
             area
California  423967
Texas       695662
New York    141297
Florida     170312
Illinois    149995
```

Zwróć uwagę, że zachodzi istotna różnica w interpretacji, zachowaniu, a przede wszystkim w typie danych jednokolumnowej **ramki danych** (obiektu **dwuwymiarowego** klasy `DataFrame`) i **serii** (obiektu **jednowymiarowego** klasy `Series`):

```
print(type(df_area))
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
print(type(area))
```

```
<class 'pandas.core.series.Series'>
```

Ćwiczenie 20: Tworzenie ramki danych z tablicy *NumPy*
Mając dwuwymiarową tablicę danych *NumPy* o trzech wierszach i dwóch kolumnach, wypełnioną próbkami liczb rzeczywistych, wylosowanych z rozkładu równomiernego na przedziale [0, 1), utwórz i wyświetl obiekt `DataFrame`, nadając kolumnom nazwy `foo` i `bar`, zaś indeksom wierszy – `a`, `b` oraz `c`. Zastosuj metodę `np.random.rand()`, pamiętając o wcześniejszym zaimportowaniu pakietu *NumPy* (`import numpy as np`).

```
print(pd.DataFrame(np.random.rand(3, 2), columns=['foo', 'bar'],
                   index=['a', 'b', 'c']))
```

```
        foo       bar
a  0.567779  0.038893
b  0.609965  0.340398
c  0.675906  0.290133
```

Ćwiczenie 21: Tworzenie ramki danych z tablicy *NumPy* (c.d.)
Skonstruuj ramkę danych jak w poprzednim ćwiczeniu, nazywając ją `df` i pomijając argumenty `columns` oraz `index` konstruktora. Jednocześnie nadaj danym w ramce typ `float32`. Wyświetl samą ramkę oraz informacje o niej.

```
df = pd.DataFrame(np.random.rand(3, 2), dtype='float32')
print(df)
```

```
          0         1
0  0.348398  0.156695
1  0.348550  0.689951
2  0.494751  0.115574
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3 entries, 0 to 2
Data columns (total 2 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   0       3 non-null      float32
 1   1       3 non-null      float32
dtypes: float32(2)
```

Zwróć uwagę, że pominięcie któregokolwiek z wyżej wymienionych argumentów konstruktora spowodowało, że dla odpowiadającego mu wymiaru (kolumn lub wierszy) został zastosowany domyślny indeks całkowity. Podobnie, jak dla konstruktora klasy `Series`, również w tym wypadku można – używając argumentu `dtype` – wymusić typ danych w ramce (ale jednolity dla wszystkich jej pól).

Ćwiczenie 22: Tworzenie ramki danych z list i wektorów *NumPy*
Zbuduj i wyświetl ramkę danych `states`, bazując na danych jak w poprzednich ćwiczeniach – potraktuj tym razem listę nazw stanów jako dodatkową kolumnę o nazwie `state`, a nie jako indeks wierszy. W tym celu jako argument konstruktora klasy `DataFrame` użyj słownika, zawierającego jako klucze nazwy kolumn, zaś jako odpowiadające im wartości – równej długości listy (lub jednowymiarowe tablice *NumPy* dla danych numerycznych), wypełnione odpowiednimi informacjami.

```
dict = {'state': ['California', 'Texas', 'New York', 'Florida', 'Illinois'],
        'population': np.array([38332521, 26448193, 19651127, 19552860, 12882135]),
        'area': np.array([423967, 170312, 149995, 141297, 695662])}
states = pd.DataFrame(dict)
print(states)
```

```
        state  population    area
0  California    38332521  423967
1       Texas    26448193  170312
2    New York    19651127  149995
3     Florida    19552860  141297
4    Illinois    12882135  695662
```

Jak widać, powstałej ramce danych został automatycznie przypisany domyślny indeks wierszy (podobnie jak w przypadku serii) a kolumny zostały nazwane tak, jak klucze w słowniku i rozmieszczone zgodnie z kolejnością ich podawania.

Możliwe jest także dodawanie nowych kolumn do istniejącej ramki danych – również wówczas, gdy została ona utworzona jako pusty obiekt `DataFrame` lub nie zawiera definicji nowej kolumny – a także tworzenie kolumn na bazie już istniejących, np. poprzez obliczenia arytmetyczne.

Ćwiczenie 23: Tworzenie nowej kolumny w ramce danych
Utwórz pustą ramkę danych o nazwie `states`, zawierającą kolumny `state` i `population` a następnie wypełnij te kolumny odpowiednimi danymi, bazując na listach wartości, jak poprzednio. Wyświetl otrzymaną ramkę.

```
states = pd.DataFrame(columns=['state', 'population'])
states['state'] = ['California', 'Texas', 'New York', 'Florida', 'Illinois']
states['population'] = [38332521, 26448193, 19651127, 19552860, 12882135]
print(states)
```

```
        state  population
0  California    38332521
1       Texas    26448193
2    New York    19651127
3     Florida    19552860
4    Illinois    12882135
```

Ćwiczenie 24: Wyliczanie wartości w kolumnie ramki danych

Do ramki danych z poprzedniego ćwiczenia dodaj kolumnę `area`, wypełniając ją danymi z listy, jak w poprzednich ćwiczeniach. Następnie wylicz gęstość zaludnienia w nowej kolumnie o nazwie `density`, dzieląc przez siebie wartości z kolumn `population` i `area`. Wyświetl otrzymaną ramkę danych.

```
states['area'] = [423967, 170312, 149995, 141297, 695662]
states['density'] = states['population'] / states['area']
print(states)
```

```
        state  population    area      density
0  California    38332521  423967    90.413926
1       Texas    26448193  170312   155.292598
2    New York    19651127  149995   131.011880
3     Florida    19552860  141297   138.381282
4    Illinois    12882135  695662    18.517807
```

Ćwiczenie 25: Przekształacanie kolumny ramki danych w indeks

Nadaj ramce danych `states`, otrzymanej w poprzednim ćwiczeniu, jawny indeks wierszy, określony za pomocą nazw poszczególnych stanów z kolumny `state`. Wypisz otrzymaną ramkę danych na standardowe wyjście.

Należy użyć metody `set_index()`, podając jako argument kolumnę, która stanie się indeksem ramki danych. Zwróć uwagę, że podany w tej metodzie argument `inplace` o wartości `True` spowoduje, że zmodyfikuje ona oryginalną ramkę danych, zamiast (domyślnie) tworzyć jej kopię:

```
states.set_index('state', inplace=True)
print(states)
```

```
            population    area      density
state
California    38332521  423967    90.413926
Texas         26448193  170312   155.292598
New York      19651127  149995   131.011880
Florida       19552860  141297   138.381282
Illinois      12882135  695662    18.517807
```

Zwróć uwagę, że tak powstały indeks przejął nazwę kolumny, z której został utworzony. Nic w tym dziwnego – posiada on atrybut `name` (podobnie jak obiekt `Series`) i można go użyć w momencie tworzenia indeksu za pomocą konstruktora.

Ćwiczenie 26: Przekształacanie indeksu w kolumnę ramki danych

Postąp odwrotnie, niż w ćwiczeniu poprzednim, by z istniejącego indeksu w powyższej ramce danych `states` uczynić kolumnę (przywracając tym samym domyślny indeks). Wyświetl tę ramkę danych.

Należy użyć metody `reset_index()`:

```
states.reset_index(inplace=True)
print(states)
```

```
        state  population    area      density
0  California    38332521  423967    90.413926
1       Texas    26448193  170312   155.292598
2    New York    19651127  149995   131.011880
3     Florida    19552860  141297   138.381282
4    Illinois    12882135  695662    18.517807
```

W przypadku, gdy takiej konwersji dokonujemy dla nienazwanego indeksu, odpowiednia kolumna zostanie nazwana po prostu `index`. Można wówczas użyć argumentu `names`, aby nazwać indeks zgodnie z naszą wolą. Natomiast użycie argumentu `drop` o wartości `True` spowoduje, że indeks zostanie zresetowany do domyślnego indeksu całkowitego, ale do ramki danych nie zostanie dodana nowa kolumna z wcześniejszymi wartościami indeksu.

Ćwiczenie 27: Przekształacanie indeksu w kolumnę ramki danych (c.d.)
Utwórz kopię ramki danych z ćwiczenia 16, przekształcając jej indeks na kolumnę. Wyświetl tak otrzymaną ramkę danych.

```
states1 = states.reset_index()
print(states1)
```

```
        index  population    area
0  California    38332521  423967
1       Texas    26448193  695662
2    New York    19651127  141297
3     Florida    19552860  170312
4    Illinois    12882135  149995
```

Ćwiczenie 28: Usuwanie indeksu ramki danych
Usuń indeks ramki danych z ćwiczenia 16, bez tworzenia jej kopii. Wyświetl otrzymaną ramkę danych.

```
states.reset_index(drop=True, inplace=True)
print(states)
```

```
   population    area
0    38332521  423967
1    26448193  695662
2    19651127  141297
3    19552860  170312
4    12882135  149995
```

Ćwiczenie 29: Tworzenie ramek danych na bazie tablic strukturyzowanych *NumPy*
Używając jako wzorca tablicy strukturyzowanej *NumPy*, utwórz i wyświetl ramkę danych o trzech wierszach i dwóch kolumnach, składającą się z samych zer. Pierwszą kolumnę nazwij `A` i nadaj jej typ całkowity 8-bajtowy, drugą zaś `B`, nadając jej typ zmiennoprzecinkowy 8-bajtowy.

Wprowadzone w pakiecie *NumPy* tzw. tablice strukturyzowane (będące w istocie protoplastami ramek danych) mogą wprost posłużyć do utworzenia odpowiadającego im obiektu `DataFrame`:

```
print(pd.DataFrame(np.zeros(3, dtype=[('A', 'i8'), ('B', 'f8')])))
```

```
   A    B
0  0  0.0
1  0  0.0
2  0  0.0
```

Zwróć uwagę, iż zastosowane powyżej w metodzie `np.zeros()` – tworzącej złożoną z samych zer tablicę *NumPy* – argumenty, specyfikujące nazwy i typy danych kolumn tejże tablicy, zostały wykorzystane przez konstruktor `DataFrame` do określenia nazw i typów danych kolumn ramki danych.

2.3. Indeksy

Powyższe rozważania pozwoliły nam stwierdzić, że zarówno obiekty `Series`, jak i `DataFrame` zawierają jawne indeksy, które umożliwiają odwoływanie się do danych i modyfikowanie ich. Owe obiekty klasy `Index` są same w sobie interesującą strukturą – można je traktować jako niemutowalne (niemodyfikowalne) tablice lub uporządkowane zbiory (technicznie rzecz biorąc, wielozbiory, ponieważ obiekty `Index` mogą zawierać powtarzające się wartości). To spojrzenie ma ciekawe konsekwencje w operacjach dostępnych na obiektach `Index`.

Ćwiczenie 30: Tworzenie indeksu na bazie listy
Skonstruuj indeks o nazwie `ind` z listy liczb całkowitych `[2, 3, 5, 7, 11]` a następnie wypisz go na ekranie.

```
ind = pd.Index([2, 3, 5, 7, 11])
print(ind)
```

```
Index([2, 3, 5, 7, 11], dtype='int64')
```

Indeks pod wieloma względami działa jak tablica, co pozwala użyć standardowej notacji indeksowania *Pythona* do pobrania jego pojedynczych wartości lub fragmentów.

Ćwiczenie 31: Dostęp do elementów indeksu
Wyświetl pierwszy element indeksu `ind` z poprzedniego ćwiczenia oraz co drugi jego element.

```
print(ind[1])
```

```
3
```

```
print(ind[::2])
```

```
Index([2, 5, 11], dtype='int64')
```

Ćwiczenie 32: Atrybuty indeksu
Wypisz na ekranie rozmiar, kształt, liczbę wymiarów oraz typ danych indeksu `ind` z ćwiczenia 30.

Obiekty klasy `Index` mają również szereg atrybutów, znanych z tablic *NumPy*:

```
print(ind.size, ind.shape, ind.ndim, ind.dtype)
```

```
5 (5,) 1 int64
```

Ćwiczenie 33: Niemutowalność indeksów
Spróbuj pierwszemu elementowi indeksu `ind` z ćwiczenia 30 nadać wartość 0.

Różnica między obiektami klasy `Index` a tablicami *NumPy* polega na tym, że indeksy są niemutowalne, co oznacza, że nie można ich modyfikować w normalny sposób:

```
ind[1] = 0
```

```
TypeError: Index does not support mutable operations
```

Ta niezmienność sprawia, że współdzielenie indeksów pomiędzy wieloma różnymi ramkami danych czy seriami jest bezpieczniejsze, bez potencjalnych skutków ubocznych wynikających z niezamierzonej modyfikacji indeksu. Z tego właśnie powodu, aby zmienić indeks serii bądź ramki danych[14], należy użyć metody `reindex()`.

Ćwiczenie 34: Tworzenie serii z indeksem jawnym
Stwórz od nowa serię danych `data`, bazując na liście `[0.25, 0.5, 0.75, 1.0]` i nadając jej elementom indeksy z listy `['c', 'b', 'f', 'd']`:

```
data = pd.Series([0.25, 0.5, 0.75, 1.0], index=['c', 'b', 'f', 'd'])
print(data)
```

```
c    0.25
b    0.50
f    0.75
d    1.00
dtype: float64
```

Ćwiczenie 35: Nadawanie serii nowego indeksu
Nadaj serii danych z poprzedniego ćwiczenia nowy indeks `['a', 'b', 'c', 'd', 'e', 'f']`. Zapamiętaj kopię serii jako `new_data` i wyświetl jej zawartość.

Metoda `reindex()` domyślnie zwraca **kopię** oryginalnej serii bądź ramki danych, z nowymi indeksami – jest to podyktowane faktem, że nowy obiekt zazwyczaj ma różny od oryginalnego zestaw wierszy bądź kolumn. Jedynie w przypadku, gdy nowy indeks jest permutacją istniejącego, można użyć argumentu `copy` z wartością `False`:

```
new_data = data.reindex(index=['a', 'b', 'c', 'd', 'e', 'f'])
print(new_data)
```

```
a     NaN
b    0.50
c    0.25
d    1.00
e     NaN
f    0.75
dtype: float64
```

Jak widać, wpisy indeksu, które nie były obecne w oryginalnej ramce danych („a” oraz „e”), zostały domyślnie wypełniane wartościami `NaN` (ang. *Not-a-Number*), oznaczającymi w tym przypadku brak danych. W razie potrzeby możemy uzupełnić brakujące wartości – o tym jednak powiemy przy okazji prezentacji metod, służących przygotowywaniu danych do uczenia maszynowego.

3. Konwencje i sposoby indeksowania

Zarówno dla serii, jak i ramek danych, pakiet *pandas* wprowadza pewne – często mylące i nieintuicyjne – konwencje dostępu do elementów, czy fragmentów tych struktur. Przyjrzyjmy się więc kilku nieoczywistym sposobom ich indeksowania.

Jak już wiemy, zarówno obiekty `Series`, jak i `DataFrame` mogą mieć zdefiniowane indeksy jawne, określone bądź w chwili ich tworzenia, bądź później – za pomocą zaprezentowanych już tutaj metod `set_index()` i `reindex()`. Nie przeszkadza to jednak (zazwyczaj) odwoływać się do składowych ramek, czy serii, za pomocą indeksów niejawnych.

[14] Dla ramki danych dostępny jest też argument `columns`, służący reindeksowaniu nazw kolumn.

Ćwiczenie 36: Indeksowanie ramek danych
Dla ramki danych z ćwiczenia 25, zawierającej informacje o stanach USA, ich powierzchni oraz liczby ludności, wyświetl na ekranie informację o powierzchni stanu New York. Skorzystaj zarówno z indeksów jawnych, jak i niejawnych (domyślnych).

Aby wykonać ćwiczenie, musielibyśmy najpierw sięgnąć do kolumny „area", a następnie wydobyć z niej element, odpowiadający indeksowi `New York`:

```
print(states['area']['New York'])
```

```
149995
```

Równie dobrze jednak – korzystając z indeksu niejawnego dla wierszy (serii)[15] – moglibyśmy napisać (pamiętając o tym, że indeksy domyślne zaczynają się od zera):

```
print(states['area'][2])
```

```
149995
```

Przypomina to trochę (odwróconą – w sensie podawania najpierw indeksu kolumny, a potem wiersza) konwencję indeksowania dwuwymiarowych tablic, znaną z takich języków, jak C/C++, czy Java i jest zgodne z ideą traktowania ramek danych jako słownika (kolumn), składającego się ze słowników (wierszy). Gdybyśmy jednak zastosowali – wydawałoby się, oczywistą – notację, jak dla tablic *NumPy*:

```
print(states['New York', 'area'])
```

otrzymalibyśmy błąd, wynikający z braku wspomnianego już wyżej indeksatora:

```
KeyError: ('New York', 'area')
```

Wspomniana już wcześniej nieintuicyjność konwencji indeksowania ramek danych polega m.in. na tym, że podając pojedynczy indeks lub ich listę odwołujemy się do kolumn, natomiast stosując *slicing*, odnosimy się do indeksów wierszy (pamiętając o tym, że górna granica zakresu – w przypadku indeksów jawnych – brana jest inkluzywnie).

Ćwiczenie 37: Wybieranie listy kolumn z ramki danych
Z ramki danych `states` z ćwiczenia 25 wybierz kolumny, zawierające informacje o liczbie ludności i gęstości zaludnienia, wyświetlając je na ekranie.

Użyjemy indeksowania ramki danych listą potrzebnych nam nazw kolumn. Zwróć uwagę na „podwójne" nawiasy kwadratowe – zewnętrzna para służy indeksowaniu ramki danych, wewnętrzna zaś ogranicza listę nazw kolumn, z niej wybieranych:

```
print(states[['population', 'density']])
```

```
            population      density
state
California    38332521    90.413926
Texas         26448193   155.292598
New York      19651127   131.011880
Florida       19552860   138.381282
Illinois      12882135    18.517807
```

[15] Ale tylko dla serii – w przypadku odwołania się za pomocą indeksu domyślnego do kolumny ramki danych, trzeba użyć tzw. indeksatora, o czym dalej.

Ćwiczenie 38: Wybieranie zakresu wierszy z ramki danych
Z ramki danych `states` z ćwiczenia 25 wybierz kolumny, zawierające informacje o stanach USA od Teksasu po Florydę.

Pamiętamy, że chcąc wybrać zakres elementów listy, czy serii, stosujemy *slicing*. Podobnie czynimy dla ramki danych – jednak inaczej, niż w poprzednim ćwiczeniu, w tym wypadku odnosimy się do indeksów wierszy, a nie kolumn:

```
print(states['Texas':'Florida'])
```

```
          population    area      density
state
Texas       26448193  170312   155.292598
New York    19651127  149995   131.011880
Florida     19552860  141297   138.381282
```

Co więcej, można wziąć wycinek ramki danych, stosując numeryczne indeksy niejawne – wówczas górna granica zakresu brana jest ekskluzywnie:

```
print(states[1:4])
```

```
          population    area     density
state
Texas       26448193  170312  155.292598
New York    19651127  149995  131.011880
Florida     19552860  141297  138.381282
```

3.1. Indeksatory

Z pomocą w tym zamieszaniu przychodzą tzw. indeksatory: `iloc` oraz `loc`. Ten pierwszy zawsze odnosi się do indeksów domyślnych, definiowanych niejawnie w sposób automatyczny, zaś drugi – do indeksów jawnych (o ile zostały określone – jeśli nie, to do indeksów domyślnych). Nie tylko pozwala to w sposób bardziej jednolity (w sensie konwencji, stosowanej dla tablic *NumPy*) zapisywać indeksy elementów ramek danych, do których chcemy uzyskać dostęp, ale też uniknąć nieporozumień przy indeksowaniu, czy wręcz umożliwić pewne jego sposoby – np. w sytuacji, gdy zdefiniujemy własne (jawne) indeksy o wartościach numerycznych (całkowitych) i przesłonią one indeksy domyślne (niejawne).

Ćwiczenie 39: Użycie indeksatorów do ramki danych
Używając indeksatorów `loc` oraz `iloc`, wybierz z ramki danych `states` informację i powierzchni stanu New York i wyświetl ją na ekranie.

```
print(states.loc['New York', 'area'])
print(states.iloc[2, 1])
```

```
149995
149995
```

Wygoda użycia indeksatorów przejawia się także w sytuacjach, gdy zechcemy w bardziej złożony sposób odwoływać się do fragmentów ramek danych – np. jednocześnie stosując *slicing* dla wierszy i wybierając określone (listą) kolumny.

Ćwiczenie 40: Użycie indeksatorów do ramki danych (c.d.)
Używając indeksatorów `loc` oraz `iloc`, wybierz z ramki danych `states` z ćwiczenia 25 informacje o liczbie ludności i gęstości zaludnienia dla stanów od Teksasu po Florydę.

Używając indeksatora `loc` i indeksów jawnych (pamiętajmy o tym, że indeks górny dla wycinków brany jest w tej sytuacji inkluzywnie) rozwiązanie ćwiczenia będzie wyglądało następująco:

```
print(states.loc['Texas':'Florida', ['population', 'density']])
```

```
          population      density
state
Texas       26448193   155.292598
New York    19651127   131.011880
Florida     19552860   138.381282
```

Ten sam przykład z użyciem `iloc` i indeksów niejawnych (pamiętajmy o tym, że indeks górny dla wycinków brany jest w tej sytuacji ekskluzywnie) będzie wyglądał następująco:

```
print(states.iloc[1:4, [0, 2]])
```

```
          population      density
state
Texas       26448193   155.292598
New York    19651127   131.011880
Florida     19552860   138.381282
```

Jak widać, stosując indeksatory, w sposób spójny podajemy – podobnie, jak dla tablic w pakiecie *NumPy*, czy innych, powszechnie używanych językach programowania – najpierw indeksy wiersza, a potem kolumny. Indeksatory pozwalają też w prosty sposób dodać nowy wiersz do istniejącej ramki danych (lub – ogólnie – uzyskać dostęp do pojedynczego wiersza).

Ćwiczenie 41: Dodawanie nowego wiersza do ramki danych
Dodaj do ramki danych `states` z ćwiczenia 25 informacje o stanie Alaska, mającgo liczbę ludności równą 738430 i powierzchnię 665384. Wyświetl zmodyfikowaną ramkę danych.

```
states.loc['Alaska'] = {'population': 738430, 'area': 665384,
                        'density': 738430 / 665384}
print(states)
```

```
            population     area     density
state
California    38332521   423967   90.413926
Texas         26448193   170312  155.292598
New York      19651127   149995  131.011880
Florida       19552860   141297  138.381282
Illinois      12882135   695662   18.517807
Alaska          738430   665384    1.109780
```

Indeksatory pozwalają też uniknąć problemów z tzw. indeksowaniem łańcuchowym (ang. *chained indexing*), czyli sytuacją, w której stosujemy indeksy do wyniku poprzedniej operacji indeksowania (patrz pierwszy przykład w tym podrozdziale). Ponieważ *pandas* może w tym wypadku zwrócić zarówno tzw. widok (inaczej zwany płytką kopią), jak i pełną kopię ramki danych (oddzielną instancję tych samych danych) – w zależności od układu w pamięci podstawowego obiektu `DataFrame` – trudno jest przewidzieć końcowy wynik operacji indeksowania a tym samym tego, czy nowa wartość zostanie wpisana do oryginalnej ramki danych, czy do jej tymczasowej kopii. Podczas wykonywania kodu, w takiej sytuacji pojawia się zazwyczaj komunikat „SettingWithCopyWarning: ...", zalecający użycie indeksatora `loc` do wyboru miejsca w ramce danych, któremu ma być nadana nowa wartość.

3.2. Filtrowanie danych

Wyobraźmy sobie, że chcemy z powyższej ramki danych wybrać wiersze, spełniające określone kryteria. Użyjemy tu tzw. masek, tworzonych za pomocą warunków logicznych – najczęściej zapisywanych za pomocą operatorów relacji arytmetycznych („`<`”, „$\leq$”, „`>`”, „$\geq$”, „`==`” i „$\neq$”) oraz spójników logicznych – negacji, alternatywy (zwykłej i rozłącznej) oraz koniunkcji.

Ćwiczenie 42: Tworzenie masek logicznych
Dla ramki danych `states` z ćwiczenia 41 utwórz maskę logiczną `mask`, bazującą na kryterium gęstości zaludnienia dla poszczególnych stanów większej niż 100 osób na milę kwadratową (w takich jednostkach podana jest bowiem powierzchnia stanów). Wyświetl otrzymaną maskę.

Weźmy w tym celu kolumnę „density” i porównajmy jej wartości z liczbą[16] 100:

```
mask = states['density'] > 100
print(mask)
```

```
state
California    False
Texas          True
New York       True
Florida        True
Illinois      False
Alaska        False
Name: density, dtype: bool
```

Otrzymaliśmy, w dość naturalny sposób, serię wartości logicznych (boolowskich), które mówią nam, czy odpowiedni wiersz w ramce danych zawiera w danej kolumnie wartości spełniające zadany warunek.

Ćwiczenie 43: Wybór wierszy z ramki danych za pomocą maski logicznej
Użyj maski logicznej z poprzedniego ćwiczenia do wybrania z ramki danych `states` z ćwiczenia 41 wierszy, spełniających warunek gęstości zaludnienia większej niż 100 osób na milę kwadratową. Wyświetl wynik na ekranie.

Mimo to, iż w powyższej masce pojawia się pojedynczy indeks, to dotyczy on właśnie wierszy, a nie kolumn. Nic dziwnego – maska jest bowiem serią, o tych samych indeksach, co ramka danych `states`, na bazie której ją utworzyliśmy. Możemy więc za jej pomocą z – teraz już całej – ramki danych `states`, wybrać te wiersze, które odpowiadają pozycjom wartości `True` w masce:

```
print(states[mask])
```

```
          population    area     density
state
Texas       26448193  170312  155.292598
New York    19651127  149995  131.011880
Florida     19552860  141297  138.381282
```

Nic nie stoi na przeszkodzie, by połączyć indeksowanie maską z innymi, poznanymi już wcześniej, sposobami – najlepiej stosując indeksator `loc`.

Ćwiczenie 44: Indeksowanie ramki danych za pomocą złożonych wyrażeń logicznych
Zbuduj złożone wyrażenie logiczne, realizujące (jak powyżej) kryterium gęstości zaludnienia większej niż 100 osób na milę kwadratową, dodając przy tym warunek, że liczba ludności w wybranym stanie

[16] A w zasadzie z serią liczb, każda o wartości 100, o długości takiej, jak liczba elementów w kolumnie „density” – zadziałała tu zasada tzw. broadcastingu, którą pakiet *pandas* „dziedziczy” po pakiecie *NumPy*.

nie może przekroczyć 20 mln, jednocześnie zakładając, że interesują nas kolumny `population` i `area`. Wyświetl elementy ramki danych `states` z ćwiczenia 41, spełniające tak sformułowany warunek.

```
print(states.loc[(states['density'] > 100) & (states['population'] ≤ 20e6),
                 ['population', 'area']])
```

```
          population    area
state
New York    19651127  149995
Florida     19552860  141297
```

Nie tworzyliśmy tym razem odrębnej zmiennej, przechowującej maskę, lecz – jak to się zwykle czyni – zindeksowaliśmy wiersze naszej ramki danych, maską tworzoną „w locie". Zwróć ponadto uwagę, że – ponieważ maski logiczne w *pandasie* są w istocie maskami **bitowymi** – musieliśmy użyć operatora koniunkcji **bitowej** „`&`", zamiast koniunkcji logicznej „`and`". Jako że operatory bitowe mają wyższy priorytet, niż operatory relacji, należało też użyć nawiasów. Trzeba pamiętać, że operatory logiczne wykonują pojedyncze wyznaczenie wartości logicznej **całego** obiektu, podczas gdy operatory bitowe („`&`", „`|`" i „`^`") wykonują wyliczenia wartości logicznych dla **poszczególnych elementów** (bit po bicie lub bajt po bajcie) obiektu. W przypadku tablic boolowskich w panadasie, czy *NumPy* ta druga opcja daje (prawie zawsze) pożądany wynik.

4. Operowanie danymi

W tym rozdziale przedstawimy podstawowe rodzaje operacji na danych, zapisanych w formie obiektów `DataFrame`, w tym czytanie danych z plików różnych formatów i zapisywanie do nich zawartości ramek danych, a także szereg innych funkcji pakietu *pandas* oraz metod klas `Series` i `DataFrame`, służących do codziennej pracy z danymi, w celu ich przygotowania do procesu uczenia maszynowego.

4.1. Operacje plikowe

Odczytywanie danych i udostępnianie ich (często nazywane ładowaniem danych) to niezbędny pierwszy krok, aby móc korzystać z większości narzędzi w ramach niniejszego kursu. Często do opisania procesu ładowania danych tekstowych i interpretowania ich jako np. tabel (czy też innych typów danych) używa się terminu parsowanie (ang. *parsing*). Dalej skoncentrujemy się na wprowadzaniu i wyprowadzaniu danych za pomocą funkcji i metod *pandasa* – jednak również w innych bibliotekach znajduje się wiele narzędzi, pomagających w czytaniu i zapisywaniu danych w różnych formatach.

4.1.1. Pliki CSV

Pakiet *pandas* oferuje szereg funkcji do wczytywania danych tabelarycznych jako obiektu `DataFrame`. Jedną z najczęściej używanych jest `read_csv()`, która pozwala czytać dane z plików tekstowych w formacie określonym jako CSV (ang. *Comma Separated Values*) – bądź zgromadzonych na dysku, bądź dostępnych poprzez URL w zasobach Internetu.

Ze względu na to, jak bardzo nieuporządkowane potrafią być dane w prawdziwym świecie, funkcja ta, wraz z rozwojem pakietu *pandas*, zgromadziła długą listę opcjonalnych argumentów – można się wręcz poczuć przytłoczonym ich liczbą, gdyż obecnie jest ich prawie pół setki. Dokumentacja *on-line*[17], jak również witryna *StackOverflow*[18], zawierają wiele przykładów działania każdego z nich – jeśli więc natkniesz się na trudności z odczytaniem konkretnego pliku, być może ktoś inny napotkał podobny problem i uda Ci się odszukać wystarczająco podobny przykład, który pomoże Ci znaleźć odpowiednie argumenty.

[17] https://pandas.pydata.org/docs/reference/api/pandas.read_csv.html
[18] https://stackoverflow.com/

Ponieważ pliki CSV są czysto tekstowe, specyfikacja typów danych kolumn nie jest częścią tego formatu danych. Oznacza to, że – poza pewnymi wyjątkami kolumn numerycznych (rzeczywistych lub całkowitych), logicznych czy łańcuchowych, dla których `read_csv()` radzi sobie dość dobrze z ich rozpoznaniem – będziemy musieli określić te typy. Uwagi może też wymagać obsługa dat i innych typów niestandardowych.

Ćwiczenie 45: Tworzenie standardowego pliku CSV
Utwórz na dysku (za pomocą dowolnego edytora tekstowego) standardowy plik CSV o nazwie `ex1.csv` i zawartości, jak poniżej, którego pierwszy wiersz stanowi nagłówek z (oddzielonymi przecinkami) nazwami kolumn, zaś kolejne wiersze zawierają dane, również z poszczególnymi polami oddzielonymi przecinkami.

```
a,b,c,d,message
1,2.1,3,True,hello
5,6.5,7,False,world
9,10.9,11,True,foo
```

Ćwiczenie 46: Czytanie zawartości pliku CSV do ramki danych
Użyj funkcji `read.csv()` z domyślnymi argumentami, w celu wczytania zawartości pliku `ex1.csv` do ramki danych `df`. Wyświetl otrzymaną ramkę oraz typy danych w jej kolumnach, oddzielając te informacje wierszem, złożonym z myślników.

```
df = pd.read_csv('ex1.csv')
print(df)
print('——————————————————————————')
print(df.dtypes)
```

```
   a     b   c      d message
0  1   2.1   3   True   hello
1  5   6.5   7  False   world
2  9  10.9  11   True     foo
——————————————————————————
a            int64
b          float64
c            int64
d             bool
message     object
dtype: object
```

Jak widać, zawartość pliku została poprawnie wczytana, zinterpretowana (co do typów numerycznych i boolowskich) i umieszczona w ramce danych, której nazwy kolumn zostały pobrane z nagłówka pliku. Pewien problem występuje tylko z kolumną `message`, której został nadany typ `object`, stojący na samym szczycie hierarchii typów *Pythona*, a więc najogólniejszy z możliwych. Jest on używany przez funkcję `read_csv()` do typów nierozpoznanych (w tym łańcuchów znakowych).

Ćwiczenie 47: Modyfikacja typu danych
Skonwertuj dane w kolumnie `message` powyższej ramki danych na typ `string`. Wyświetl typy danych w tak zmodyfikowanej ramce.

Uściślenia typu danych `object`, na typ konkretny dokonujemy za pomocą odpowiedniego rzutowania (już po wczytaniu ramki danych), za pomocą metody `astype()`:

```
df['message'] = df['message'].astype('string')
print(df.dtypes)
```

```
a            int64
b          float64
c            int64
d             bool
message     string
dtype: object
```

Ćwiczenie 48: Specyfikacja typu danych
Wyspecyfikuj typ danych w kolumnie `message` jako `string` w trakcie wczytywania powyższej ramki danych. Wyświetl typy danych w tak wczytanej ramce.

Aby jawnie wymusić zinterpretowanie typu danych w kolumnie pliku w odpowiedni sposób już w trakcie jego wczytywania (licząc się jednak z możliwością wystąpienia błędu), użyjemy argumentu `dtype` funkcji `read_csv()`:

```
df = pd.read_csv('ex1.csv', dtype={'message': 'string'})
print(df.dtypes)
```

```
a            int64
b          float64
c            int64
d             bool
message     string
dtype: object
```

Ćwiczenie 49: Tworzenie niestandardowego pliku CSV
Utwórz na dysku (za pomocą dowolnego edytora tekstowego) plik CSV o nazwie `ex2.csv` i zawartości, jak poniżej – jednak nie zawierającego wiersza nagłówka, a wyłącznie wiersze z danymi, z poszczególnymi polami oddzielonymi przecinkami.

```
1,2.1,3,True,hello
5,6.5,7,False,world
9,10.9,11,True,foo
```

Ćwiczenie 50: Czytanie zawartości pliku CSV bez nagłówka do ramki danych
Użyj funkcji `read.csv()` z argumentem `header=None`, w celu wczytania zawartości pliku `ex2.csv` do ramki danych `df`. Wyświetl otrzymaną ramkę danych.

```
df = pd.read_csv('ex2.csv', header=None)
print(df)
```

```
0  1   2.1   3   True   hello
1  5   6.5   7  False   world
2  9  10.9  11   True     foo
```

Ćwiczenie 51: Czytanie zawartości pliku CSV bez nagłówka do ramki danych (c.d.)
Użyj funkcji `read.csv()` z odpowiednimi argumentami, w celu wczytania zawartości pliku `ex2.csv` do ramki danych `df` w taki sposób, by jej kolumnom nadać nazwy „a”, „b”, „c”, „d” oraz „message” i potraktować tę ostatnią jako indeks ramki danych. Wyświetl otrzymaną ramkę danych.

Tym razem musimy za pomocą argumentu `names` funkcji `read_csv()` określić listę nazw kolumn tworzonej ramki danych, jednocześnie za pomocą argumentu `index_col` specyfikując kolumnę `message` jako źródło indeksu ramki danych. W innym wypadku pierwszy wiersz pliku może zostać (błędnie) zinterpretowany jako wiersz nagłówka, lub – w najlepszym razie – zamiast nazw kolumn w naszej ramce danych będziemy mieli niejawne indeksy numeryczne.

```
df = pd.read_csv('ex2.csv', names=[ 'a', 'b', 'c', 'd', 'message'], index_col='message')
print(df)
```

```
         a     b   c      d
message
hello    1   2.1   3   True
world    5   6.5   7  False
foo      9  10.9  11   True
```

Jedną z częściej używanych opcji przy czytaniu plików CSV jest ustawienie znaku separatora kolumn – zwłaszcza, że np. pliki tego typu, wyeksportowane z arkuszy MS Excel, domyślnie zawierają w tym miejscu średnik, zamiast przecinka (ze względu z kolei na stosowanie m. in. w Polsce tego ostatniego jako separatora dziesiętnego[19]). Odpowiedzialny jest za to argument `sep` (jego aliasem jest `delimiter`) funkcji `read_csv()`, który pozwala również na definiowanie tzw. wyrażeń regularnych – np. w sytuacji, gdy czytany plik jest w formacie o stałej szerokości kolumn, oddzielonych od siebie pewną liczbą tzw. białych znaków (spacji lub tabulatorów)[20].

Ćwiczenie 52: Tworzenie pliku tekstowego
Utwórz na dysku (za pomocą dowolnego edytora tekstowego) plik o nazwie `ex3.txt` i zawartości, jak poniżej.

```
A         B         C
aaa -0.264438 -1.026059 -0.619500
bbb  0.927272  0.302904 -0.032399
ccc -0.264273 -0.386314 -0.217601
ddd -0.871858 -0.348382  1.100491
```

Ćwiczenie 53: Czytanie zawartości pliku tekstowego z użyciem wyrażeń regularnych
Użyj funkcji `read.csv()` z odpowiednim argumentem, specyfikującym wyrażenie regularne, w celu wczytania zawartości pliku `ex3.txt` do ramki danych `df`. Wyświetl otrzymaną ramkę danych.

Wyrażenie regularne (ang. *Regular Expression* – *regex*) `'\s+'`, którego użyjemy, jest wzorcem, pasującym do co najmniej jednego białego znaku:

```
df = pd.read_csv('ex3.txt', sep='\s+')
print(df)
```

```
            A         B         C
aaa -0.264438 -1.026059 -0.619500
bbb  0.927272  0.302904 -0.032399
ccc -0.264273 -0.386314 -0.217601
ddd -0.871858 -0.348382  1.100491
```

Ponieważ w pierwszym wierszu pliku, zawierającym nazwy kolumn, był o jeden element mniej, niż liczba kolumn w wierszach danych, funkcja `read_csv()` „wywnioskowała”, że w tym szczególnym przypadku pierwsza kolumna powinna być potraktowana jako indeks obiektu `DataFrame`.

Obsługa wartości brakujących jest ważną i często wymagającą dopracowania częścią procesu odczytu pliku. Brakujące dane zwykle albo nie występują (pusty ciąg znaków między separatorami pól), albo są oznaczone jakąś wartością wskaźnikową (symbolem zastępczym, tzw. wartownikiem – ang. *sentinel*). Domyślnie *pandas* używa powszechnie występujących wartowników, takich jak *NA* i *NULL*.

[19] Za jego ustawienie odpowiada argument `decimal`, zaś za separator, oddzielający kolejne 3-cyfrowe grupy (np. tysiące od setek) w liczbach – argument `thousands` funkcji `read_csv()`.

[20] W tym wypadku można też użyć dedykowanych do tego rodzaju plików `funkcji read_fwf()` lub `read_table()`.

Ćwiczenie 54: Tworzenie pliku CSV z pustymi wartościami i wczytywanie go do ramki danych
Utwórz na dysku (za pomocą dowolnego edytora tekstowego) plik o nazwie `ex4.csv` i zawartości, jak poniżej.

```
something,a,b,c,d,message
one,1,2,3,4,NA
two,5,6,,8,world
three,NULL,10,11,12,foo
```

Nastepnie wczytaj ten plik do ramki danych `df` i wyświetl ją na ekranie.

```
df = pd.read_csv('ex4.csv')
print(df)
```

```
  something    a   b     c   d message
0       one  1.0   2   3.0   4     NaN
1       two  5.0   6   NaN   8   world
2     three  NaN  10  11.0  12     foo
```

Jak widac, efektem wczytania pliku jest ramka danych, zawierająca wartości brakujące, oznaczone jako `NaN` (ang. *Not-a-Number* – „nieliczba"). Jest to specjalna wartość, zdefiniowana zgodnie ze standardem IEEE w typie danych `float64` pakietu NumPy – z tego też powodu kolumna `a` została potraktowana jako zawierająca wartości zmiennoprzecinkowe, bowiem tylko w tym typie możliwe jest reprezentowanie wartości `NaN`.

Ćwiczenie 55: Zapisywanie danych obiektu `DataFrame` do pliku CSV
Za pomocą metody `to_csv()` klasy `DataFrame` zapisz ramkę danych `df` do pliku o nazwie `ex4_out.csv`. Otwórz ten plik w dowolnym edytorze tekstowym i przeanalizuj jego zawartość.

```
df.to_csv('ex4_out.csv')
```

Zapisany na dysku plik będzie miał następującą zawartość:

```
,something,a,b,c,d,message
0,one,1.0,2,3.0,4,
1,two,5.0,6,,8,world
2,three,,10,11.0,12,foo
```

Jak widać, został domyślnie utworzony niejawny (numeryczny) indeks wierszy, krórego wartości pojawiły się jako pierwsza kolumna w pliku – za wyjątkiem wiersza nagłówka, gdzie brak wartości przed pierwszym przecinkiem jest interpretowany jako wartość pusta (`NaN`), czyli brak danych.

Ćwiczenie 56: Zapisywanie danych obiektu `DataFrame` do pliku CSV bez indeksu wierszy
Za pomocą metody `to_csv()` klasy `DataFrame` z odpowiednimi argumentami zapisz ramkę danych `df` do pliku o nazwie `ex4_out.csv` w taki sposób, by nie zawierał on niejawnych (numerycznych) indeksów wierszy. Otwórz ten plik w dowolnym edytorze tekstowym i przeanalizuj jego zawartość.

```
df.to_csv('ex4_out.csv', index=False)
```

Zapisany na dysku plik będzie miał następującą zawartość – podobną, jak oryginalny, źródłowy plik `ex4.csv` (dane brakujące zostaną domyślnie wypisane jako pusta wartość między przecinkami):

```
something,a,b,c,d,message
one,1.0,2,3.0,4,
two,5.0,6,,8,world
three,,10,11.0,12,foo
```

4.1.2. Inne formaty

Do czytania/pisania plików z/do skoroszytów w formatach *MS Excel* i analogicznych (pliki XLS, XLSX, XLSM, XLSB, ODF, ODS oraz ODT) służy odpowiednio funkcja *pandasa* `read_excel()` oraz metoda `to_excel()` klasy `DataFrame`. Domyślnie wczytują/zapisują one pierwszy arkusz ze skoroszytu – można jednak użyć argumentu `sheet_name`, aby za pomocą nazwy lub numeru arkusza wyspecyfikować dowolny z nich. Pozostałe opcje – a jest ich wiele – opisano w ich dokumentacji[21], zawierającej też wiele typowych przykładów użycia.

Z kolei do operacji na plikach JSON (ang. *JavaScript Object Notation*) służą odpowiednio – do czytania funkcje `read_json()`[22] z *pandasa* oraz `load()` z pakietu *json*[23] (wymaga wcześniejszego zaimportowania), zaś do pisania – metoda `to_json()`[24] klasy `DataFrame` oraz funkcja `dump()` z pakietu *json*. Format JSON stał się jednym ze standardowych formatów przesyłania danych za pomocą żądania HTTP pomiędzy przeglądarkami internetowymi a innymi aplikacjami. Jest to znacznie bardziej swobodny format danych niż tabelaryczny formularz tekstowy, taki jak CSV. Jednak jego omówienie (a tym samym i funkcji/metod, służących do jego przetwarzania) wykracza poza ramy niniejszego kursu – zainteresowanych zachęcamy więc do sięgnięcia do dokumentacji i zamieszczonych w niej przykładów.

Python ma wiele bibliotek do odczytu i zapisu danych we wszechobecnych formatach HTML i XML. Przykłady obejmują *lxml*, *Beautiful Soup* i *html5lib*. Chociaż *lxml* jest ogólnie znacznie szybszy, inne biblioteki radzą sobie lepiej ze źle sformatowanymi plikami HTML lub XML. Pakiet *pandas* z kolei posiada wbudowaną funkcję `read_html()`[25], która wykorzystuje wszystkie te biblioteki do automatycznego analizowania tabel z plików HTML i wczytywania ich jako obiektów `DataFrame`. Szczegółowy opis jej użycia, wraz z przykładami, podaje m in. Wes McKinney w cytowanej już wcześniej książce „Python for Data Analysis" – podobnie zresztą jak i metod tzw. serializacji danych, czyli zapisu ich (oraz odczytu) w różnorodnych formatach binarnych (*pickle*, HDF5), pracy z *Web API*, czy bazami danych. Znowu, ich dyskusja wykraczałaby (tym razem znacznie) poza zakres niniejszego skryptu – zainteresowani mogą szerzej studiować wybrane zagadnienia na bazie powszechnie dostępnej w Internecie dokumentacji, samouczków, przykładów, czy innych kursów.

4.2. Wybrane funkcje i metody

W tym podrozdziale krótko opiszemy wybrane funkcje pakietu *pandas* oraz metody klas `Series` i `DataFrame`, niezbędne do realizacji zadań kursu.

- Metody `head()`[26] oraz `tail()`[27] – zwracają (odpowiednio) domyślnie pięć pierwszych/ostatnich wierszy serii lub ramki danych. Liczbę tę można określić argumentem `n`.

Ćwiczenie 57: Wyświetlanie pierwszych/ostatnich wierszy ramki danych
Dla ramki danych `states` z ćwiczenia 41 wyświetl trzy pierwsze wiersze danych, a następnie ostatnie jej wiersze, w domyślnej liczbie.

```
print(states.head(n=3))
```

[21] https://pandas.pydata.org/docs/reference/api/pandas.read_excel.html oraz https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.to_excel.html
[22] https://pandas.pydata.org/docs/reference/api/pandas.read_json.html
[23] https://docs.python.org/3/library/json.html
[24] https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.to_json.html
[25] https://pandas.pydata.org/docs/reference/api/pandas.read_html.html
[26] https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.head.html – analogiczne metody istnieją dla obiektów Series, referencje do nich łatwo znaleźć w dokumentacji.
[27] https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.tail.html

```
            population    area     density
state
California    38332521  423967   90.413926
Texas         26448193  170312  155.292598
New York      19651127  149995  131.011880
```

```
print(states.tail())
```

```
          population    area     density
state
Texas       26448193  170312  155.292598
New York    19651127  149995  131.011880
Florida     19552860  141297  138.381282
Illinois    12882135  695662   18.517807
Alaska        738430  665384    1.109780
```

- Metoda `copy()`[28] tworzy kopię serii lub ramki danych – jest ona szczególnie przydatna w sytuacji, gdy operujemy na wycinku struktury, który w istocie jest widokiem na tę strukturę i współdzieli z nią dane – modyfikacja wycinka spowodowałaby tym samym zmianę oryginalnych danych, co nie zawsze jest pożądane.

Ćwiczenie 58: Tworzenie kopii ramki danych

Utwórz kopię fragmentu ramki danych `states` z ćwiczenia 41, zawierającą informacje z kolumn od `population` do `area` dla stanów od New York do Illinois, nazywając ją `states_copy`. Wypisz jej zawartość na standardowe wyjście.

```
states_copy = states.loc['New York':'Illinois', 'population':'area'].copy()
print(states_copy)
```

```
          population    area
state
New York    19651127  149995
Florida     19552860  141297
Illinois    12882135  695662
```

Ćwiczenie 59: Modyfikacja danych kopii

Zmodyfikuj dane obiektu `states_copy`, dotyczące liczby ludności dla Florydy, podając wartość 99999999. Następnie wyświetl na ekranie zawartość tej ramki danych, a następnie oryginalnej ramki danych `states`.

```
states_copy.loc['Florida', 'population'] = 99999999
print(states_copy)
```

```
          population    area
state
New York    19651127  149995
Florida     99999999  141297
Illinois    12882135  695662
```

```
print(states)
```

[28] https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.copy.html

```
            population    area     density
state
California    38332521  423967   90.413926
Texas         26448193  170312  155.292598
New York      19651127  149995  131.011880
Florida       19552860  141297  138.381282
Illinois      12882135  695662   18.517807
Alaska          738430  665384    1.109780
```

Nie spowodowało to – jak widać powyżej – zmiany w oryginalnej ramce danych `states`, na bazie której stworzyliśmy wycinek `states_copy`.

Ćwiczenie 60: Tworzenie wycinka ramki danych

Utwórz wycinek ramki danych `states_copy` z poprzedniego ćwiczenia (bezpośrednio, tzn. jako widok, bez użycia metody `copy()`), wybierając informacje od Florydy po Illinois, nazywając go `states_view`. Wypisz jego zawartość na standardowe wyjście.

```
states_view = states_copy.loc['Florida':'Illinois', :]
print(states_view)
```

```
          population    area
state
Florida     99999999  141297
Illinois    12882135  695662
```

Ćwiczenie 61: Operacje na wycinku ramki danych

Przywróć pierwotną wartość (19552860) wcześniej zmienionej (w ćwiczeniu 59) komórki ramki danych `states_view`. Wypisz zawartość zarówno ramki danych, będącej wycinkiem (`states_view`), jak i pierwotnej ramki danych (`states_copy`). Przeanalizuj wynik i wyciągnij wnioski – zwróć uwagę na komunikat z ostrzeżeniem[29], pojawiający się na ekranie.

```
states_view.loc['Florida', 'population'] = 19552860
print(states_view)
```

```
          population    area
state
Florida     19552860  141297
Illinois    12882135  695662
```

Jak widać poniżej, modyfikacja danych w ramce-wycinku `states_view`, który jest „widokiem” na fragment oryginalnej ramki danych[30] spowoduje zmianę zawartości także ramki danych `states_copy`, na bazie której stworzyliśmy ów wycinek.

```
print(states_copy)
```

```
          population    area
state
New York    19651127  149995
Florida     19552860  141297
Illinois    12882135  695662
```

[29] SettingWithCopyWarning: A value is trying to be set on a copy of a slice from a DataFrame

[30] Czyli – formalnie rzecz biorąc – referencją do tego samego obszaru pamięci, który jest zajmowany przez oryginalny obiekt `states_copy`.

- Zadaniem funkcji `concat()`[31] jest łączenie (sklejanie) ze sobą serii i ramek danych.

Ćwiczenie 62: Tworzenie przykładowych ramek danych
Utwórz dwie przykładowe dwie ramki danych (nazwij je `df1` i `df2`) – pierwsza niech ma 3 wiersze (nazwane `a`, `b` i `c`) oraz 2 kolumny (nazwane `one` i `two`), zawierające kolejne liczby całkowite od 0 do 5, druga zaś – 2 wiersze (nazwane `a` i `c`) oraz 2 kolumny (nazwane `two` i `six`), zawierające kolejne liczby całkowite od 5 do 8. Wypisz te ramki na ekranie, oddzielając je linią, składającą się z myślników.

```
df1 = pd.DataFrame(np.arange(6).reshape(3, 2), index=['a', 'b', 'c'],
                   columns=['one', 'two'])
df2 = pd.DataFrame(5 + np.arange(4).reshape(2, 2), index=['a', 'c'],
                   columns=['two', 'six'])
print(df1)
print('-----------')
print(df2)
```

```
   one  two
a    0    1
b    2    3
c    4    5
-----------
   two  six
a    5    6
c    7    8
```

Ćwiczenie 63: Łączenie wierszowe ramek danych
Połącz powyższe ramki wiersz po wierszu, korzystając z metody `concat()`. Wypisz wynik na ekranie.

Przekazując do metody `concat()` listę ramek oraz argument `axis` o wartości `'index'` (wartość domyślna – równoważnie można użyć argumentu `axis=0`) spowodujemy wierszowe łączenie ramek danych:

```
print(pd.concat([df1, df2], axis='index'))
```

```
   one  two  six
a  0.0    1  NaN
b  2.0    3  NaN
c  4.0    5  NaN
a  NaN    5  6.0
c  NaN    7  8.0
```

Ćwiczenie 64: Łączenie kolumnowe ramek danych
Połącz powyższe ramki kolumna po kolumnie, korzystając z metody `concat()`. Wypisz wynik na ekranie.

Przekazując do metody `concat()` listę ramek oraz argument `axis` o wartości `'columns'` (równoważnie można użyć argumentu `axis=1`) spowodujemy kolumnowe łączenie ramek danych:

```
print(pd.concat([df1, df2], axis='columns'))
```

```
   one  two  two  six
a    0    1  5.0  6.0
b    2    3  NaN  NaN
c    4    5  7.0  8.0
```

[31] https://pandas.pydata.org/docs/reference/api/pandas.concat.html

Zwróć uwagę, że nastąpiło tu zjawisko, zwane dopasowaniem indeksów – ponieważ w ramce `df1` brak kolumny o nazwie `six`, zaś w ramce `df2` kolumny o nazwie `one` – w ramce wynikowej, powstałej z łączenia wierszowego, w odpowiadających im częściach kolumn pojawiły się wartości puste (`NaN`). Analogicznie, ponieważ w ramce `df2` brak wiersza o indeksie `c`, w wynikowej ramce danych, powstałej z łączenia kolumnowego, wartości puste zostały wstawione do odpowiednich części wierszy[32]. Jako efekt uboczny wstawienia wartości NaN widzimy konwersję danych do typu `float64`, który jest w stanie te wartości reprezentować. Ponadto pojawiły się – zazwyczaj niepożądane – duplikaty indeksów.

Ćwiczenie 65: Powielone indeksy ramek danych
Z ramki danych, otrzymanej w ćwiczeniu 63 wybierz wartości, odpowiadające wierszom o indeksie `a`. Wypisz wynik na ekranie i przeanalizuj go.

```
print(pd.concat([df1, df2]).loc['a', :])
```

```
   one  two  six
a  0.0    1  NaN
a  NaN    5  6.0
```

Ćwiczenie 66: Łączenie ramek danych bez powielania indeksów
Połącz ramki danych, jak w ćwiczeniu 63, unikając powielenia indeksów. Wynikową ramkę danych nazwij `df3` i wypisz jej zawartość na ekranie.

Do metody `concat()` należy przekazać argument `ignore_index` z wartością `False` – wówczas istniejące indeksy zostaną pominięte i zostanie utworzony nowy numeryczny indeks niejawny (domyślny):

```
df3 = pd.concat([df1, df2], ignore_index=True)
print(df3)
```

```
   one  two  six
0  0.0    1  NaN
1  2.0    3  NaN
2  4.0    5  NaN
3  NaN    5  6.0
4  NaN    7  8.0
```

Ćwiczenie 67: Dodawanie pojedynczego wiersza do ramki danych
Do ramki danych `df1` z ćwiczenia 62 dodaj wiersz o indeksie `d`, zawierajacy w kolumnie `one` wartość 7, zaś w kolumnie `two` – wartość 8. Wyświetl tak zmodyfikowaną ramkę danych.

Zadanie można zrealizować w prosty sposób bez użycia metody `concat()` – w przypadku, gdy ma ona indeks jawny, wystarczy postąpić analogicznie, jak w przypadku dodawania nowych kolumn (patrz ćwiczenia 24 i 41) – trzeba jednak użyć indeksatora `loc`:

```
df1.loc['d'] = {'one': 7, 'two': 8}
print(df1)
```

```
   one  two
a    0    1
b    2    3
c    4    5
d    7    8
```

[32] Używając argumentu `join` z wartością `'inner'`, spowodujemy, że w wynikowej ramce danych znajdą się tylko te wiersze (kolumny), których indeksy (nazwy) są wspólne dla obu łączonych ramek. Opisane w tekście działanie metody concat() jest zachowaniem domyślnym, odpowiadającym wartości `'outer'` argumentu `join`.

W sytuacji, gdy ramka ma wyłącznie niejawny indeks numeryczny musimy wyliczyć jego wartość dla dodawanego wiersza – będzie ona równa liczbie istniejących już wierszy (bo te numerujemy od zera).

Ćwiczenie 68: Dodawanie pojedynczego wiersza do ramki danych (c.d.)
Do ramki danych `df3` z ćwiczenia 66 dodaj nowy wiersz, zawierajacy w kolumnie `one` wartość 6, zaś w kolumnie `two` – wartość 9. Wyświetl tak zmodyfikowaną ramkę danych.

Indeks nowo wstawianego wiersza będzie o jeden większy, niż indeks ostatniego wiersza w ramce danych, który jest o jeden mniejszy niż liczba już istniejących wierszy w ramce danych (gdyż numerujemy je od zera). Pamiętając, że funkcja `len` zwraca liczbę elementów struktury na najwyższym poziomie, możemy napisać:

```
df3.loc[len(df3)] = {'one': 6, 'two': 9}
print(df3)
```

```
   one  two  six
0  0.0    1  NaN
1  2.0    3  NaN
2  4.0    5  NaN
3  NaN    5  6.0
4  NaN    7  8.0
5  9.0   10  NaN
6  6.0    9  NaN
```

Zwróć przy okazji uwagę, że brak w słowniku, opisującym nowy wiersz, klucza dla kolumny `six` w ramce danych, spowodował nadanie temu polu wartości pustej (`NaN`).

Ćwiczenie 69: Wstawianie nowej kolumny do ramki danych
Do ramki danych `df3` z ćwiczenia powyżej dodaj tuż po kolumnie `two` kolumnę o nazwie `five`, zawierającą wartości z listy `[9, 8, 7, 6, 5, 4, 3]`. Wynik wypisz na ekranie.

Jeżeli chcemy wstawić do istniejącej ramki danych nową kolumnę, lecz nie jako ostatnią a w ściśle określonej pozycji, należy użyć metody `insert()`[33]. Jej działanie jest intuicyjne – podajemy jako argumenty kolejno: indeks (numeryczny) pozycji, przed którą zostanie wstawiona nowa kolumna (argument `loc` – pierwszy w kolejności), jej nazwę (argument `column` – drugi w kolejności) oraz definicję – listę wartości[34], tablicę lub serię (argument `value` – trzeci w kolejności):

```
df3.insert(2, 'five', [9, 8, 7, 6, 5, 4, 3])
print(df3)
```

```
   one  two  five  six
0  0.0    1     9  NaN
1  2.0    3     8  NaN
2  4.0    5     7  NaN
3  NaN    5     6  6.0
4  NaN    7     5  8.0
5  9.0   10     4  NaN
6  6.0    9     3  NaN
```

Przy okazji zademonstrowaliśmy, jak używać argumentów funkcji i metod w formie tzw. pozycyjnej (ang. *positional arguments*), znając ich kolejność w definicjach tychże – bez konieczności podawania nazw. Oczywiście, przy większej liczbie argumentów, sposób tradycyjny (poprzez tzw. argumenty kluczowe – ang. *keyword arguments*), choć bardziej rozwlekły, jest mniej podatny na błędy.

[33] https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.insert.html
[34] Możliwe jest też podanie wartości stałej, wtedy nowa kolumna zostanie nią w całości wypełniona.

- Do usuwanie wierszy lub kolumn służy metoda `drop()`[35].

Ćwiczenie 70: Usuwanie wierszy z ramki danych
Z ramki danych `states` z ćwiczenia 41 usuń wiersz, zawierajacy dane dla Teksasu. Nie twórz kopii ramki danych, lecz zmodyfikuj jej oryginalną postać. Wynik wypisz na ekranie.

```
states.drop('Texas', inplace=True)
print(states)
```

```
            population    area     density
state
California    38332521  423967   90.413926
New York      19651127  149995  131.011880
Florida       19552860  141297  138.381282
Illinois      12882135  695662   18.517807
Alaska          738430  665384    1.109780
```

Użyty argument inplace z wartością `True` powoduje, że zmiany dokonują się na oryginalnym obiekcie `DataFrame`, bezpośrednio w obszarze pamięci, w którym jest on przechowywany. Pominięcie tego argumentu (jego domyślna wartość wynosi `False`) spowodowałoby, zwrócenie przez metodę `drop()` zmodyfikowanej kopii oryginalnej ramki. Pominęliśmy również argument `axis`, gdyż jego domyślna wartość 0 (lub – równoważnie – `'index'`) oznacza, że operacja usuwania dotyczy indeksów wierszy. Alternatywnie moglibyśmy wyspecyfikować indeksy usuwanych wierszy, używając argumentu `index` i nadając mu wartość etykiety (lub listy etykiet) usuwanych wierszy.

Ćwiczenie 71: Usuwanie kolumn z ramki danych
Z ramki danych `states` z poprzedniego ćwiczenia usuń kolumny `population` i `density`. Tym razem stwórz kopię ramki danych, nazywając ją `states_area` i wyświetl jej zawartość na ekranie.

Chcąc usunąć kolumnę (lub większą ich liczbę) należy wyspecyfikować ich nazwy jako wartość argumentu `columns`, lub użyć argumentu `axis` z wartością 1 (lub równoważnie `'columns'`):

```
states_area = states.drop(columns=['population', 'density'])
print(states_area)
```

```
              area
state
California  423967
New York    149995
Florida     141297
Illinois    695662
Alaska      665384
```

Celowo pominęliśmy tu argument `inplace`, by pokazać tworzenie kopii obiektu `DataFrame`.

- Metoda `rename()`[36] służy zmianie nazwy obiektu `Series`, `DataFrame`, bądź `Index`. Najczęściej jednak będziemy używać jej w stosunku do ramki danych. Określenie tego, czy modyfikujemy etykiety wierszy, czy nazwy kolumn, odbywa się podobnie[37], jak dla opisanej powyżej metody `drop()`. Wyspecyfikować jedynie musimy – poprzez stojący na pierwszej pozycji – argument mapper, definiując w formie słownika lub funkcji odwzorowanie istniejących etykiet na nowe. Możemy też użyć definicji tego odwzorowania jako wartości argumentu `index` lub `columns`.

[35] https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.drop.html
[36] https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.rename.html
[37] Identyczna też jest rola argumentu `inplace`.

Ćwiczenie 72: Zmiana nazw kolumn w ramce danych
W ramce danych `states` z ćwiczenia 70 zmień nazwę kolumny `populaton` na `inhabitants` (bez tworzenia kopii ramki). Wynik wypisz na ekranie.

```
states.rename(columns={'population': 'inhabitants'}, inplace=True)
print(states_area)
```

```
            inhabitants    area     density
state
California     38332521  423967   90.413926
New York       19651127  149995  131.011880
Florida        19552860  141297  138.381282
Illinois       12882135  695662   18.517807
Alaska           738430  665384    1.109780
```

- Jeżeli chcielibyśmy zmienić typ danych w serii lub kolumnie (kolumnach) ramki danych[38], należy użyć metody `astype()`[39].

Ćwiczenie 73: Zmiana typów danych w ramce danych
W ramce danych z poprzedniego ćwiczenia zmień typ danych w kolumnie `area` na `float64`. Utwórz kopię oryginalnej ramki, nazywając ją `states_real`. Wypisz na ekranie jej zawartość oraz typy danych, oddzielając owe informacje wierszem, składającym się z myślników.

Skoro powierzchnię stanu możemy podać z dowolną dokładnością – stosując ułamek dziesiętny, a nie tylko liczby całkowite – zapewne wolelibyśmy, by w powyższej ramce danych kolumna `area` miała typ rzeczywisty (zmiennoprzecinkowy). Zrobimy to zatem w następujący sposób:

```
states_real = states.astype({'area': 'float'})
print(states_real)
print('-----------------------------------------')
print(states_real.dtypes)
```

```
            inhabitants      area     density
state
California     38332521  423967.0   90.413926
New York       19651127  149995.0  131.011880
Florida        19552860  141297.0  138.381282
Illinois       12882135  695662.0   18.517807
Alaska           738430  665384.0    1.109780
-----------------------------------------
inhabitants      int64
area           float64
density        float64
dtype: object
```

Wprawdzie metoda `astype()` posiada argument `copy` (z domyślną wartością `True`), którego znaczenie w zasadzie można by interpretować jako odwrotne do znaczenia argumentu `inplace` innych metod, jednak dokumentacja zaleca ostrożność w jego używaniu – nie zawsze działa on zgodnie z oczekiwaniami i jest ignorowany, w przypadku, gdy operacja zmiany typu nie może być zrealizowana bezpośrednio na oryginalnej tablicy.

[38] Operację tę określa się często mianem **rzutowania** typów.
[39] https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.astype.html

- Zastosowanie metody `transpose()`[40] jest równoważne[41] użyciu atrybutu `T` obiektu `DataFrame` i oznacza przestawienie (zamianę) wierszy i kolumn.

Ćwiczenie 74: Transponowanie ramki danych
Wypisz na ekranie transpozycję ramki danych `states_copy` z ćwiczenia 61.

```
print(states_copy.T)
```

```
state       New York   Florida  Illinois
population  19651127  19552860  12882135
area          149995    141297    695662
```

- Jedną z mocnych stron *Pythona* jest względna łatwość obsługi i manipulowania danymi tekstowymi. Opierając się na tym, *pandas* zapewnia kompleksowy zestaw operacji zwektoryzowanych[42] na ciągach znaków, które stają się niezbędnym elementem podczas pracy z danymi ze świata rzeczywistego. Upraszcza to składnię operacji na seriach i ramkach danych: nie musimy się już martwić ich rozmiarem ani kształtem, ale tylko tym, jaką operację chcemy wykonać. Jeśli dobrze rozumie się manipulację ciągami w *Pythonie*, większość składni operacji dla ciągów w *pandasie* jest na tyle intuicyjna, że wystarczy – zapominając o pętlach – po prostu sięgnąć do spisu dostępnych metod[43]. Ich nazwy są poprzedzone prefiksem `str` i oddzielone kropką, wskazując tym samym na fakt, że operują nie na serii danych jako całości, a na poszczególnych (wszystkich) jej elementach.

Ćwiczenie 75: Manipulowanie danymi tekstowymi w ramce
Z ramki danych `states` z ćwiczenia 24 wybierz i wypisz na standardowe wyjście te wiersze, w których nazwa stanu kończy się na literę „a”.

```
print(states[states['state'].str[-1] == 'a'])
```

```
        state  population    area     density
0  California    38332521  423967   90.413926
3     Florida    19552860  141297  138.381282
```

Użyte tu wyrażenie `str[i]` jest równoważne metodzie `str.get(i)` i pozwala na dostęp do *i*-tego elementu listy, krotki, słownika lub łańcucha znaków, dla każdej ze składowych obiektu `Series`, na rzecz którego zostało ono użyte.

Ćwiczenie 76: Manipulowanie danymi tekstowymi w ramce (c.d.)
Z ramki danych `states` z ćwiczenia 24 wybierz i wypisz na standardowe wyjście te wiersze, w których nazwa stanu zawiera ciąg znaków „or”.

Użyjemy metody `str.contains()`:

```
print(states[states['state'].str.contains('or')])
```

```
        state  population    area     density
0  California    38332521  423967   90.413926
2    New York    19651127  149995  131.011880
3     Florida    19552860  141297  138.381282
```

Ćwiczenie 77: Zamiana wielkości liter tekstów w ramce danych
W ramce danych `states` z ćwiczenia 24 zamień wszystkie litery w nazwach stanów na wielkie. Wynik wyświetl na ekranie.

[40] https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.transpose.html
[41] Jeśli nie chcemy określać dodatkowych argumentów – np. `copy`.
[42] Czyli takich, które wymagają przetworzenia każdego elementu serii danych (wektora) w ten sam sposób.
[43] https://pandas.pydata.org/docs/reference/series.html

Użyjemy metody `str.upper()`:

```
states['state'] = states['state'].str.upper()
print(states)
```

```
        state  population    area     density
0  CALIFORNIA    38332521  423967   90.413926
1       TEXAS    26448193  170312  155.292598
2    NEW YORK    19651127  149995  131.011880
3     FLORIDA    19552860  141297  138.381282
4    ILLINOIS    12882135  695662   18.517807
```

Jak widać, działanie zwektoryzowanych metod łańcuchowych w *pandasie* jest takie, jak odpowiadających im nazwą metod łańcuchowych w *Pythonie*.

- Do pracy z danymi typu data/czas (czyli znacznikami czasowymi) służą atrybuty oraz metody, które posiadają prefiks `dt`.

Ćwiczenie 78: Manipulowanie danymi typu data/czas w ramce
Utwórz ramkę danych `df`, zawierającą kolumnę `date` o wartościach typu `datetime64`, odpowiadającym datom z listy `['2021-01-01', '2022-06-30', '2023-12-31']` oraz kolumnę `value`, zawierającą wartości numeryczne z listy `[0.286920, 0.611632, 0.478675]`. Wypisz zawartość ramki danych na standardowe wyjście.

```
dates = ['2021-01-01', '2022-06-30', '2023-12-31']
df = pd.DataFrame({'date': pd.to_datetime(dates),
    'value': [0.286920, 0.611632, 0.478675]})
print(df)
```

```
        date     value
0 2021-01-01  0.286920
1 2022-06-30  0.611632
2 2023-12-31  0.478675
```

Ćwiczenie 79: Manipulowanie danymi typu data/czas w ramce (c.d.)
Z kolumny `date` w ramce danych `df` z poprzedniego ćwiczenia wybierz rok i wyświetl wynik na ekranie.

Należy sięgnąć do atrybutu `dt.year` kolumny `date`:

```
print(df['date'].dt.year)
```

```
0    2021
1    2022
2    2023
Name: date, dtype: int64
```

Użyta w powyższym przykładzie funkcja *pandasa* `to_datetime()` służy do konwersji różnych formatów daty i czasu na standardowe znaczniki czasowe (obiekty klasy `datetime.datetime` *Pythona*[44] lub klasy `datetime64` pakietu *NumPy*[45]). Kluczowym jej argumentem jest format, pozwalający określić sposób interpretacji poszczególnych składowych daty i czasu w łańcuchach znaków, które podlegają konwersji. Znaczenie tzw. dyrektyw w łańcuchu formatującym można znaleźć w dokumentacji funkcji `strftime()/strptime()` *Pythona*[46].

[44] https://docs.python.org/3/library/datetime.html#datetime.datetime
[45] https://numpy.org/doc/stable/reference/arrays.datetime.html
[46] https://docs.python.org/3/library/datetime.html#strtime-and-strptime-behavior

Ćwiczenie 80: Zaawansowane manipulowanie danymi typu data/czas w ramce
Utwórz ramkę danych `df`, zawierającą kolumnę `date_time` o wartościach typu `datetime64` (znacznik czasu – ang. *timestamp*), odpowiadającym chwilom czasowym z listy `['09/11/2001 084615', '09/11/2001 090230', '09/11/2021 903745', '09/11/2001 100300']` oraz kolumnę `event`, zawierającą teksty z listy `['North Tower', 'South Tower', 'Pentagon', 'Pennsylvania']`. Pamiętaj o zastosowaniu odpowiedniego wzorca formatu danych typu data/czas oraz o wymuszeniu zignorowania błędów konwersji. Wypisz zawartość ramki danych na standardowe wyjście.

```
dt = pd.to_datetime(['09/11/2001 084615', '09/11/2001 090230', '09/11/2021 903745',
                     '09/11/2001 100300'],
format='%m/%d/%Y %H%M%S', errors='coerce')
df = pd.DataFrame({'date_time': dt,
                   'event': ['North Tower', 'South Tower', 'Pentagon', 'Pennsylvania']})
print(df)
```

```
            date_time         event
0 2001-09-11 08:46:15   North Tower
1 2001-09-11 09:02:30   South Tower
2                 NaT      Pentagon
3 2001-09-11 10:03:00  Pennsylvania
```

Pojawiająca się w ramce danych wartość `NaT` (ang. *Not-a-Time*) jest wynikiem (zamierzonego) błędu w zapisie godziny trzeciego zdarzenia i nadania wartości `'coerce'` (konwersja danych) argumentowi errors. Użycie natomiast wartości `'raise'` skutkowałoby wygenerowaniem wyjątku, zaś wartości `'ignore'` – brakiem konwersji błędnie zapisanych elementów.

5. Zadania sprawdzające

Poniższe zadania należy wykonać z użyciem pakietu *Pandas* w wersji co najmniej 2.2.0. Oprócz zaimportowania go, należy także zaimportować pakiet *NumPy*, w wersji co najmniej 1.26.0. **Uwaga:** W przypadku użycia pakietu *NumPy* w wersji 2.0.0 lub wyższej, należy użyć pakietu *Pandas* również w nowszej wersji – co najmniej 2.2.2. Należy też zwrócić uwagę na zainstalowane środowisko języka *Python* – listę zgodnych wersji można znaleźć w dokumentacji powyższych pakietów (https://numpy.org/news/, https://pandas.pydata.org/docs/getting_started/install.html).

5.1. Podstawy pracy z ramkami danych

Zadanie 1: Tworzenie ramki danych
Mając poniższy słownik danych `data` i listę indeksów wierszy `labels`, opisujących pacjentów gabinetu weterynaryjnego, utwórz z nich ramkę danych o nazwie `animals`. Następnie wyświetl podsumowanie podstawowych informacji o tej ramce danych i jej strukturze.

```
data = {'animal': ['cat', 'cat', 'snake', 'dog', 'dog', 'cat', 'snake', 'cat', 'dog',
'dog'],
        'name': ['Daisy', 'Bella', 'Noodle', 'Charlie', 'Max', 'Molly', 'Draco', 'Kenzo',
'Milo', 'Cooper'],
        'age': [2.5, 3, 0.5, np.nan, 5, 2, 4.5, np.nan, 7, 3],
        'visits': [1, 3, 2, 3, 2, 3, 1, 1, 2, 1],
        'priority': ['yes', 'yes', 'no', 'yes', 'no', 'no', 'no', 'yes', 'no', 'no']}

labels = ['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j']
```

Zadanie 2: Wybieranie danych z ramki danych
Mając ramkę danych `animals` z poprzedniego zadania, wybierz z niej, a następnie wyświetl:
a) pierwsze 3 wiersze,
b) kolumny `'animal'` i `'age'`,
c) dane z wierszy 3, 4 i 8, zawarte w kolumnach `'visits'` i `'priority'`,
d) wiersze, w których liczba wizyt jest większa niż 2,
e) wiersze, w których zwierzęciem jest kot i ma mniej niż 4 lata,
f) wiersze, w których wiek zwierzęcia mieści się w przedziale od 2 do 4 lat (włącznie).

Zadanie 3: Operacje na wierszach i kolumnach
Mając ramkę danych `animals` z poprzedniego zadania, w podanej poniżej kolejności:
a) dodaj do niej wiersz o indeksie `'k'`, zawierający informacje o nowym zwierzęciu – psie, wabiącym się Buddy, w wieku 5.5 roku, który odwiedził gabinet 2 razy w zwykłym trybie,
b) dodaj do niej kolumnę `'price'`, zawierającą wartość 10 dla pacjentów zwykłych i 20 dla pacjentów priorytetowych,
c) dodaj do niej kolumnę `'total'`, zawierajacą sumaryczny koszt wizyt każdego z pacjentów,
d) usuń z niej kolumnę `'priority'`,
e) zmień nazwę kolumny `'animal'` na `'species'`,
f) wyświetl otrzymaną ramkę danych.

Zadanie 4: Indeksowanie ramek danych
Mając ramkę danych `animals` z poprzedniego zadania, wykonaj w podanej kolejności poniższe kroki, po każdym z nich wyświetlając otrzymany wynik:
a) przywróć ramce danych domyślny indeks całkowity, sprawiając jednocześnie, by nie została dodana nowa kolumna z wcześniejszymi wartościami indeksu,
b) utwórz indeks z kolumny `'name'`,
c) usuń z ramki danych wiersz, zawierający informacje o psie, wabiącym się Max,
d) przywróć ramce danych domyślny indeks całkowity, sprawiając jednocześnie, by z istniejącego indeksu uczynić kolumnę danych w ramce.

5.2. Operacje plikowe, konwersja danych

Zadanie 5: Zapisywanie danych do pliku CSV
Mając ramkę danych `animals` z poprzedniego zadania, wykonaj w podanej kolejności poniższe kroki:
a) zamień w kolumnie `'species'` ciągi znaków na pisane samymi wielkimi literami,
b) zapisz tę ramkę danych do pliku CSV o nazwie `'animals.csv'` – spraw, by nie zawierał on niejawnych (numerycznych) indeksów wierszy,
c) odpowiednim poleceniem systemu operacyjnego wyświetl zawartość utworzonego pliku.

Zadanie 6: Wczytywanie danych z pliku CSV
Mając plik CSV o nazwie `'weather.csv'` (możesz go pobrać stąd), wykonaj w podanej kolejności poniższe kroki:
a) wczytaj jego zawartość do ramki danych `weather`,
b) usuń z otrzymanej ramki danych kolumnę `'Location'`,
c) zmodyfikuj kolumnę `'Date'` tak, by zawierała ona standardowe znaczniki czasowe,
d) wyświetl informacje o zmodyfikowanej ramce danych i jej strukturze oraz jej ostatnich 7 wierszy.

Zadanie 7: Operacje na typach danych
Mając ramkę danych `weather` z poprzedniego zadania, wykonaj w podanej kolejności poniższe kroki:
a) przekształć dane w kolumnach `'RainToday'` i `'RainTomorrow'` na typ logiczny (`boolean`), odwzorowując wartości `'Yes'` na `True` i `'No'` na `False`,
b) przekształć dane w kolumnach, zawierających dane o kierunku wiatru, na typ kategoryczny (`category`),
c) wyświetl informacje o zmodyfikowanej ramce danych i jej strukturze,

d) wyświetl wiersze, zawierające informacje o pogodzie w pierwszej dekadzie lutego 2008,
e) wyświetl wiersze, zawierające informacje o obserwacjach, w dniach, w których wiatr o godzinie dziewiątej wiał z kierunków południowo-zachodnich.

Zadanie 8: Wczytywanie danych z pliku MS Excel

Mając skoroszyt MS Excel o nazwie `'employee.xlsx'` (możesz go pobrać stąd), wykonaj w podanej kolejności poniższe kroki:

a) wczytaj z niego arkusz `'employee_details'` do ramki danych `emp_det`, zaś arkusz `'performance'` do ramki danych `emp_perf`,
b) przekształć w tych ramkach danych kolumnę `'name'` na indeks,
c) połącz je kolumna po kolumnie do ramki danych o nazwie `employees`,
d) zamień nazwy kolumn tak, by zaczynały się od wielkiej litery,
e) wyświetl informacje o zmodyfikowanej ramce danych i jej strukturze oraz jej zawartość.

Zadanie 9: Zapisywanie danych do pliku MS Excel

Mając ramkę danych `employees` z poprzedniego zadania, wykonaj w podanej kolejności poniższe kroki:

a) utwórz z niej ramkę danych o nazwie `emp_finance`, wybierając pracowników działu finasowego, zarabiających powyżej 20000,
b) zapisz ramkę danych `emp_finance` do skoroszytu MS Excel o nazwie `'emp_finance.xlsx'`, umieszczając dane w arkuszu o nazwie `'employee_finance'`.